

Requirement Analysis – Detailed Notes

Phase 2: Requirement Analysis

Requirement Analysis is the process of identifying, documenting, validating, and managing the needs and expectations of stakeholders for a software system. It is divided into Functional Requirements and Non-Functional Requirements. The objective is to ensure that developers clearly understand what must be built and how it should perform.

Functional Requirements

Functional requirements define what a system must do. They describe system features, services, business rules, and expected behavior under specific conditions. Key Characteristics: • Describe system functionality. • Focus on user needs and business processes. • Can be tested directly through inputs and outputs. • Usually specified by users, clients, or stakeholders. Examples: • System sends a confirmation email after account creation. • System generates approval requests after user submits information. • Users can log in and manage profiles. • Customers can place orders and make payments. Common Types: • Authentication • Authorization • External Interfaces • Transaction Processing • Reporting • Business Rules • Regulatory Compliance

Listing Functional Requirements

Functional requirements can be documented in several formats: 1. Use Cases: Describe interactions between users and the system. 2. User Stories: Short requirements written from the user's perspective. Example: 'As a customer, I want to track my order so that I know its delivery status.' 3. Work Breakdown Structure (WBS): Breaks large functions into smaller manageable components. 4. Prototypes: Visual representation of system functionality. 5. Software Requirement Specification (SRS): Formal document containing complete system requirements.

Functional Requirements in Agile Workflow

Agile development organizes requirements into different levels: Initiatives: • High-level business goals. • Collection of related epics. Epics: • Large bodies of work. • Can be divided into multiple stories. User Stories: • Small requirements describing user needs. • Written from the end-user perspective. Tasks and Subtasks: • Technical implementation activities derived from stories. Benefits: • Better planning. • Incremental delivery. • Easier requirement management. • Improved team collaboration.

Non-Functional Requirements

Non-functional requirements define how a system performs rather than what it does. They specify quality attributes and operational constraints. Examples of Non-Functional Requirements: • Performance • Security • Reliability • Scalability • Availability • Maintainability • Usability Usability: • Measures ease of learning and operating the system. • Interface should be intuitive. • Accessibility requirements should be considered. Examples: • Users without specialized knowledge should operate the system easily. • Keyboard navigation should allow quick access to important functions. Security: • Protects system and data from unauthorized access. • Defines permissions and access controls. • Includes authentication, encryption, and auditing. Example: • Only administrators can modify sensitive system settings.

Functional vs Non-Functional Requirements

Functional Requirements: • Describe what the system does. • Define features and services. • Focus on user requirements. • Usually mandatory. • Tested using component, API, and UI testing. Non-Functional Requirements: • Describe how the system works. • Define quality attributes. • Focus

on user expectations. • Improve overall user experience. • Tested using performance, security, usability, and reliability testing. Examples: Functional: User registration, payment processing, report generation. Non-Functional: Fast response time, high availability, strong security, scalability.

Customer Journey Maps

Customer Journey Mapping visualizes the customer's complete experience while interacting with a product or service. Journey Components: 1. Actions – What the customer does. 2. Touchpoints – Where interactions occur. 3. Thoughts – What the customer thinks. 4. Feelings – Emotional state. 5. Ownership – Responsible teams. 6. Opportunities – Areas for improvement. Benefits: • Optimizes onboarding. • Improves customer experience. • Identifies pain points. • Helps understand customer expectations. • Improves conversion and retention. • Creates a logical view of customer interactions. Typical Stages: • Registration • Onboarding • First Session • Continued Usage • Retention

Data Flow Diagrams (DFD)

A Data Flow Diagram represents the movement of information within a system. Main Components: 1. External Entity: • Source or destination of data. • Can be a customer, department, or external system. 2. Process: • Activity that transforms data. • Converts inputs into outputs. 3. Data Store: • Repository where data is stored. • Examples: databases, files, forms. 4. Data Flow: • Movement of information between entities, processes, and data stores. • Represented using arrows. Advantages of DFD: • Easy system visualization. • Better communication with stakeholders. • Identifies bottlenecks. • Helps during system design and analysis.

Exam-Oriented Summary

Functional Requirements = WHAT the system does. Non-Functional Requirements = HOW the system performs. Agile Hierarchy: Initiative → Epic → User Story → Task → Subtask Customer Journey Map: Actions + Touchpoints + Thoughts + Feelings + Opportunities DFD Components: External Entity + Process + Data Store + Data Flow Remember: Functional requirements define features. Non-functional requirements define quality.